Big Data Technologies – Complete Cheatsheet

Sessions 20–26: Data Warehousing, ETL/ELT, Orchestration, & Apache Spark

---

SESSION 20: Introduction to Data Warehousing and Data Lakes

Data Warehouse (DW)

- **Definition**: Centralized, structured repository for integrated data from multiple sources
- **Data Type**: Structured (tables with fixed columns)
- **Schema Approach**: Schema-on-write (clean before loading)
- **Purpose**: Business reporting, dashboards, OLAP, analytics
- **Users**: Business analysts, managers, BI developers
- **Optimization**: Optimized for read-heavy analytical queries
- **Schema Design**: Star or snowflake schemas with fact and dimension tables
- **Cost**: Higher per GB, performance-optimized

Key MCQ Points: - Subject-oriented and time-variant - Uses ETL process - Supports dimensional modeling

Data Lake (DL)

- **Definition**: Large storage system holding raw data in any format
- **Data Types**: Structured, semi-structured (JSON, XML), unstructured (images, logs, audio, video)
- **Schema Approach**: Schema-on-read (interpret when reading)
- **Purpose**: Advanced analytics, ML, exploratory analysis, big data processing
- **Users**: Data engineers, data scientists, ML engineers
- **Storage**: Usually cheaper per GB
- **Scalability**: Highly scalable for big data
- **Format**: Raw files, Parquet, ORC, etc.

Key MCQ Points: - Stores data as-is - Uses ELT process - Supports diverse data types - Cost-effective for large volumes

DW vs DL Comparison

Feature	Data Warehouse	Data Lake
Data Type	Structured only	All types (structured, semi-structured, unstructured)
Schema	Schema-on-write	Schema-on-read
Purpose	Reporting & BI	Advanced analytics & ML
Users	Analysts & Managers	Data Scientists & Engineers
Cost	Higher	Lower
Data Quality	Cleaned & validated	Raw & diverse
Processing	OLAP queries	Complex transformations

SESSION 21: Designing Data Warehouses and Lakes for ETL/ELT

ETL (Extract → Transform → Load)

- **Extract**: Gather data from source systems (OLTP DBs, files, APIs)
- **Transform**: Clean, validate, join, aggregate, apply business rules in a staging/transformation layer
- **Load**: Move processed data into Data Warehouse with predefined schema
- **Schema**: Schema-on-write (schema defined before data enters warehouse)
- **Best For**: Traditional Data Warehouses, structured data, standard reports

Pipeline Stages: 1. Source Systems (OLTP, CSVs, APIs) 2. Staging Area (raw extract dump) 3. Transformation Layer (apply business rules) 4. Warehouse Storage (star/snowflake schemas)

ELT (Extract → Load → Transform)

- **Extract**: Gather raw data from sources
- **Load**: Load data immediately into Data Lake/cloud warehouse (minimal transformation)

- **Transform**: Transform inside the lake/warehouse using SQL, Spark, Python
- **Schema**: Schema-on-read (schema applied when needed)
- **Best For**: Big data, diverse data types, exploratory analysis, ML workloads

Pipeline Stages: 1. Raw Zone (landing zone with original data) 2. Cleansed/Curated Zone (transformed data) 3. Analytics/Consumer Zone (ready-to-use datasets)

When to Use ETL vs ELT

Choose ETL when: - Data is structured and well-understood - Need stable, repeatable reports - Strict data governance required - Traditional DW environment

Choose ELT when: - Data is large and diverse (logs, JSON, images) - Data scientists need raw/near-raw data - Keep option to re-transform with new logic - Cloud DW or Data Lake environment

---

SESSION 22: Airflow & Informatica – Workflow Orchestration

Apache Airflow (Open-source)

Core Concepts: - **DAG (Directed Acyclic Graph)**: Workflow blueprint defining tasks and dependencies - **Task**: Single unit of work (run script, SQL query, Spark job) - **Operator**: Template for what a task does (BashOperator, PythonOperator, SqlOperator) - **Scheduler**: Decides when tasks run based on time, dependencies, triggers - **Executor**: Actually runs tasks (LocalExecutor, CeleryExecutor, KubernetesExecutor)

Work Management: - Schedule DAGs (daily, hourly, cron expressions) - Define dependencies: `task1 >> task2` (task2 runs after task1) - Automatic retries on failure - Web UI for monitoring (status, logs, run history) - Support for complex patterns: fan-out, fan-in, branching

Use Cases: - Orchestrate multi-step ETL/ELT pipelines - Schedule and monitor jobs across different tools - Trigger downstream jobs on success/failure - Send notifications and alerts

Key MCQ Points: - General workflow orchestrator (not just ETL) - Code-based (Python DAGs) - Open-source and extensible

Informatica PowerCenter (Enterprise ETL Tool)

Core Concepts: - **Source & Target Definitions**: Metadata about where data comes from/goes - **Mapping**: Visual diagram showing data flow and trans-

formations (filter, joiner, aggregator, expression) - **Session**: Execution configuration for running a mapping - **Workflow**: Collection of sessions and tasks defining overall job flow with sequence, conditions, branches

Work Management: - Create workflows with sessions arranged in order - Define dependencies and conditions (run next step only if previous succeeds) - Schedule workflows for automated execution - Includes decision logic, timers, and event handling

Use Cases: - Traditional ETL pipelines - Data integration between enterprise systems - Heavy data transformation and movement - Governance and metadata management

Key MCQ Points: - GUI-based (drag-and-drop mappings) - Integrated ETL engine + workflow manager - Enterprise-focused

Automating Data Pipelines

With Airflow: - Define DAG with extract task → transform task → load task - Schedule to run daily at 1 AM - Airflow handles dependencies, retries, monitoring - Can coordinate Spark jobs, SQL scripts, Python code

With Informatica: - Create mappings for each logical step - Build workflow sequencing sessions - Add control logic and scheduling - Workflow runs automatically at configured times

---

SESSION 23: Apache Spark APIs – Overview & RDDs

Apache Spark Fundamentals

- **Framework**: Distributed computing for large-scale batch, SQL, streaming, ML
- **Core Idea**: Split data into partitions, process in parallel using functional style
- **SparkSession**: Main entry point (gives access to RDDs, DataFrames, SQL)
- **Driver**: Main application defining transformations/actions
- **Executors**: Run tasks on data partitions
- **Cluster Manager**: YARN, Standalone, Kubernetes, Mesos

Resilient Distributed Datasets (RDDs)

Properties: - **Resilient**: Fault-tolerant via lineage (recompute lost partitions) - **Distributed**: Split across multiple nodes - **Immutable**: Cannot change; transformations create new RDDs - **Typed**: Any object type (RDD[Int], RDD[String])

RDD Operations: - **Transformations (Lazy)**: Return new RDD - `map`: Transform each element - `filter`: Keep only matching elements - `flatMap`: Map then flatten - `reduceByKey`: Combine values by key - Examples: `rdd.map(x => x * 2)`, `rdd.filter(x => x > 0)`

- **Actions (Trigger Computation)**: Return result or write output
  - `collect()`: Return all elements to driver
  - `count()`: Count elements
  - `saveAsTextFile()`: Write to file
  - `reduce()`: Aggregate values

Creating RDDs from External Data: - `textFile("hdfs://.../file.txt")`: Read text files - `parallelize(List(1,2,3))`: From in-memory collection - Automatically partitioned; number of partitions affects parallelism

Key MCQ Points: - Transformations are lazy; actions trigger computation - Lineage enables fault tolerance - Partitioned for parallel processing

---

SESSION 24: RDD vs DataFrame vs Dataset, DataFrame Ops, Structured Streaming

RDD vs DataFrame vs Dataset

RDD (Low-level) - No schema; works with objects directly - Limited automatic optimization - Type-safe at runtime - More control, lower performance - `RDD[T]` where T is any type

DataFrame (Higher-level) - Distributed table with schema (named columns) - Backed by **Catalyst optimizer** and **Tungsten** engine - Untyped (Row-based), compile-time errors less caught - Better performance, simpler API - Similar to Pandas/SQL tables

Dataset (Scala/Java) - Typed, strongly-typed API - `Dataset[T]` where T is a class - Has schema + compile-time type safety - Combines RDD control + DataFrame optimization - Available mainly in Scala/Java

Comparison: - RDD: Low-level, control, limited optimization - DataFrame: High-level, optimized, untyped rows - Dataset: Typed DataFrame with compile-time safety

DataFrame Operations

Selection & Filtering: - `select("col1", "col2")`: Pick columns - `filter("age > 18")` or `where()`: Apply conditions

Aggregation: - `groupBy("city").agg(avg("salary"))`: Group and aggregate - `count()`, `sum()`, `avg()`, `min()`, `max()`

Joining: - `df1.join(df2, "key")`: Inner join on key - `df1.leftJoin(df2)`: Left outer join - Support for all join types

SQL Interface: - `createOrReplaceTempView("table_name")`: Register as SQL table - `spark.sql("SELECT ...")`: Write SQL queries - Mix DataFrame and SQL APIs

Key MCQ Points: - Catalyst optimizer handles query planning - Schema-based enables optimization - Can use SQL or DataFrame API

Structured Spark Streaming

Core Idea: - Treat streaming data as **unbounded table** getting new rows continuously - Reuse DataFrame/Dataset API - Spark runs queries incrementally as data arrives

Processing Models: - **Micro-batch (default)**: Process in small batches (e.g., every second) - **Continuous Processing**: Lower-latency processing (advanced)

Input Sources: - Kafka, files, sockets, rate source

Output Modes: - **Append**: Only new rows to sink - **Update**: Changed rows only - **Complete**: Entire result set

Key MCQ Points: - Uses DataFrame/Dataset API, not old DStream API - Declarative: specify what to compute, Spark manages execution - Handles windowing, stateful operations, joins

---

SESSION 25: Advanced Spark – Functions, Key-Value, Shuffle, Persistence, Shared Variables, Deployment

Passing Functions to Spark

- Functions are **serialized** from driver to executors
- Must be serializable; avoid capturing large non-serializable objects
- Examples: `map(lambda x: x * 2)`, `filter(x => x.age > 18)`

Key-Value Pairs and Shuffle

Pair RDDs: - RDDs storing **(key, value)** tuples - Enable special operations: `reduceByKey`, `groupByKey`, `aggregateByKey`, `join`, `cogroup`

Shuffle Operations: - **Shuffle**: Redistribute data across partitions so all values for a given key end up together - Causes network + disk I/O (expensive) - Operations triggering shuffle: `groupByKey`, `reduceByKey`, `repartition`, `join`, `cogroup` - Can be a performance bottleneck

Key MCQ Points: - Shuffle is expensive due to network communication - Many aggregate operations require shuffle - Understand which operations cause shuffle

RDD Persistence and Caching

Persistence: - `cache()`: Shorthand for `persist(MEMORY_ONLY)` - `persist(level)`: Choose storage level - `MEMORY_ONLY`: Keep as objects in memory (default, fastest) - `MEMORY_AND_DISK`: Spill to disk if memory full - `DISK_ONLY`: Store on disk only - `MEMORY_ONLY_SER`, `MEMORY_AND_DISK_SER`: Serialized versions

Removing Cached Data: - `unpersist()`: Free persisted data when no longer needed

Use Cases: - Reuse same RDD multiple times (avoid recomputation via lineage) - Long computations that will be queried multiple times - Balancing memory vs recomputation cost

Key MCQ Points: - Caching is lazy (happens after action) - Avoid caching unnecessary RDDs (wastes memory) - Choose right storage level based on memory/speed trade-off

Shared Variables: Broadcast & Accumulators

Broadcast Variables: - Read-only variables cached on each executor - Efficient for large lookup tables - Avoids sending same object with every task - Usage: config maps, reference tables, dictionaries

Accumulators: - Variables that tasks can **add to** (not read from tasks) - Typically for counters and aggregates - Driver can read final value after job completes - Examples: count errors, sum metrics, track events

Key MCQ Points: - Broadcast: efficient one-way distribution of data - Accumulator: aggregate values from tasks to driver - Both are optimizations for common patterns

Spark Deployment Modes

Local Mode - Everything runs on single machine - Master: `local`, `local[n]`, `local[*]` - Good for dev/testing on laptop - No cluster needed

Cluster Mode (Production) - Driver runs **inside the cluster** (typically on a worker node) - Client can disconnect after job submission - More resilient for long-running jobs - Typical for production

Client Mode - Driver runs on client/edge node - Cluster runs executors - Good for interactive work, needs client-server connectivity

Cluster Managers: - **Spark Standalone**: Spark's built-in manager - **YARN**: Hadoop's resource manager - **Kubernetes**: Container orchestration - **Mesos**: Cluster resource manager

Key MCQ Points: - Difference is mainly **where driver runs** - Cluster mode is typical for production - Local for development/testing

---

SESSION 26: MapReduce with Spark, Hadoop Integration, JDBC

MapReduce Style with Spark

Classic Word Count Pattern: 1. `flatMap` each line to words 2. `map` each word to `(word, 1)` 3. `reduceByKey(_ + _)` to count occurrences

MapReduce Phases in Spark: - **Map**: `map`, `flatMap` operations transform data - **Shuffle**: Implicit in `reduceByKey`, `groupByKey`, `join` - **Reduce**: `reduceByKey`, `aggregate`, `fold` combine values

Key Difference: - Spark doesn't use Hadoop MapReduce engine - But supports map → shuffle → reduce style via RDD/DataFrame operations - More efficient with in-memory computation

Spark with Hadoop vs without Hadoop

Spark WITH Hadoop (YARN/HDFS): - **YARN**: Acts as cluster resource manager - **HDFS**: Primary storage (data locality benefits) - Good when already have Hadoop cluster - Unified resource management with Hadoop ecosystem - Deploy: `spark-submit --master yarn`

Spark WITHOUT Hadoop (Standalone/Others): - **Standalone Manager**: Spark's own cluster manager - **Storage**: Can be local FS, cloud (S3, GCS), Cassandra, other systems - **Cluster Managers**: Kubernetes, Mesos also available - Hadoop not required; lighter-weight option - Flexible storage layer

Key Differences (MCQ Focus): - With Hadoop: YARN manages resources, HDFS is storage, integrated ecosystem - Without: Standalone or other managers, pluggable storage, can be simpler

Key MCQ Points: - Spark can run independently without Hadoop - Hadoop is optional, not mandatory - Different deployment scenarios based on infrastructure

Spark JDBC Configuration

JDBC Overview: - Spark can read/write relational databases using JDBC data source - Works via DataFrames for familiar API

Reading from Database:

```
spark.read
  .format("jdbc")
```

```
  .option("url", "jdbc:postgresql://host:port/db")
  .option("dbtable", "table_name")
  .option("user", "username")
  .option("password", "password")
  .load()
```

Writing to Database:

```
df.write
  .format("jdbc")
  .option("url", "jdbc:...")
  .option("dbtable", "table_name")
  .option("user", "username")
  .option("password", "password")
  .save()
```

Key JDBC Options: - `url`: JDBC connection string - `dbtable`: Table name or subquery - `user`, `password`: Authentication - `numPartitions`: Parallelize read/write across partitions - `partitionColumn`, `lowerBound`, `upperBound`: For parallel reads

Typical Use Cases: - Load data from SQL database into Spark for processing - Write Spark DataFrames back to relational DB - ETL pipelines connecting Spark and databases

Key MCQ Points: - JDBC is bridge between Spark DataFrames and SQL databases - Standard options: url, dbtable, user, password - Can parallelize reads/writes using partition options

---

Quick Reference: Common MCQ Patterns

Data Warehouse vs Data Lake

- **DW**: Structured, schema-on-write, reporting, analysts
- **DL**: All types, schema-on-read, exploration, scientists

ETL vs ELT

- **ETL**: Extract → **Transform** → Load (before warehouse)
- **ELT**: Extract → Load → **Transform** (inside lake)

Airflow vs Informatica

- **Airflow**: General orchestrator, code-based, open-source, DAGs
- **Informatica**: ETL-focused, GUI-based, enterprise, mappings+workflows

RDD vs DataFrame vs Dataset

- **RDD**: Low-level, no schema, manual optimization
- **DataFrame**: High-level, schema, optimized, untyped rows
- **Dataset**: Typed DataFrame, compile-time safety (Scala/Java)

Transformations vs Actions

- **Transformation**: Lazy, returns RDD/DataFrame (`map`, `filter`, `flatMap`)
- **Action**: Eager, triggers compute (`collect`, `count`, `save`)

Shuffle Operations

- Expensive due to network I/O
- Caused by: `reduceByKey`, `groupByKey`, `join`, `repartition`
- Avoid when possible; combine operations to minimize shuffles

Storage Levels

- `MEMORY_ONLY`: Fast, limited size (default)
- `MEMORY_AND_DISK`: Overflow to disk
- `DISK_ONLY`: Persistent but slow
- Use `persist()` for frequently reused RDDs

Deployment Modes

- **Local**: Dev/testing
- **Cluster**: Production (driver inside cluster)
- **Client**: Interactive (driver on client)

Shared Variables

- **Broadcast**: Distribute large read-only data efficiently
- **Accumulator**: Aggregate values from tasks (counter-like)

Important Acronyms & Terms

Term	Meaning
DW	Data Warehouse
DL	Data Lake
ETL	Extract-Transform-Load
ELT	Extract-Load-Transform
OLAP	Online Analytical Processing
OLTP	Online Transaction Processing

Term	Meaning
DAG	Directed Acyclic Graph
HDFS	Hadoop Distributed File System
YARN	Yet Another Resource Negotiator
RDD	Resilient Distributed Dataset
DF	DataFrame
JDBC	Java Database Connectivity
CSV	Comma-Separated Values
JSON	JavaScript Object Notation
Parquet	Columnar storage format
ORC	Optimized Row Columnar format

Study Tips for CCEE MCQs

1. **Understand differences**: Know when to use DW vs DL, ETL vs ELT, etc.
2. **Key concepts**: Lazy vs eager, shuffle costs, storage levels
3. **Tool matching**: Airflow for orchestration, Informatica for ETL
4. **API hierarchy**: RDD < DataFrame < Dataset (in terms of abstraction)
5. **Deployment context**: Local for dev, cluster/YARN for production
6. **Practical scenarios**: "Which operation causes shuffle?" "When to cache?" "Why broadcast?"

Last Updated: January 7, 2026
Scope: CDAC CCEE Big Data Technologies – Sessions 20–26
Format: Single-column cheatsheet for quick reference